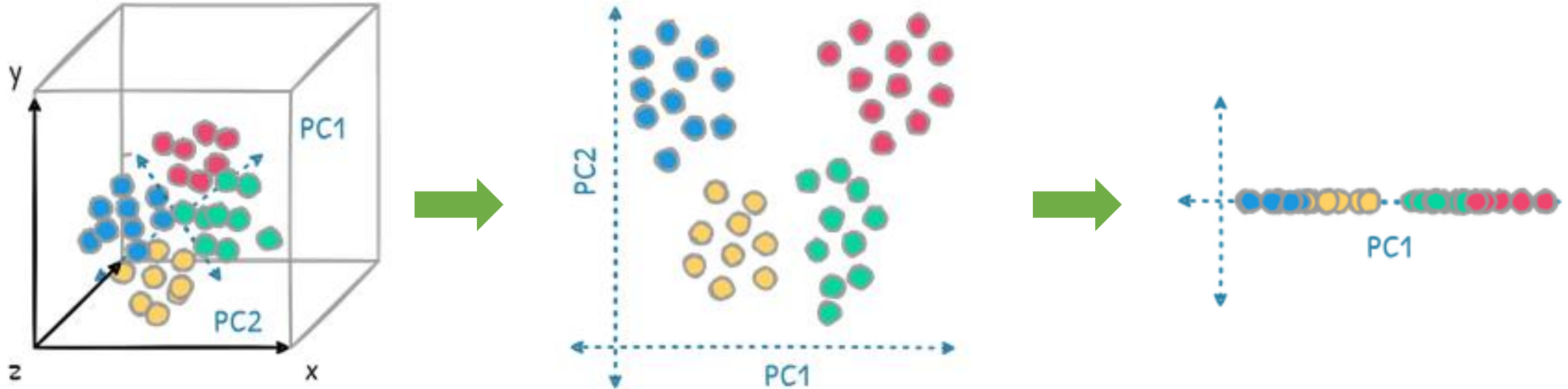


Dimensionality Reduction

Dimensionality Reduction

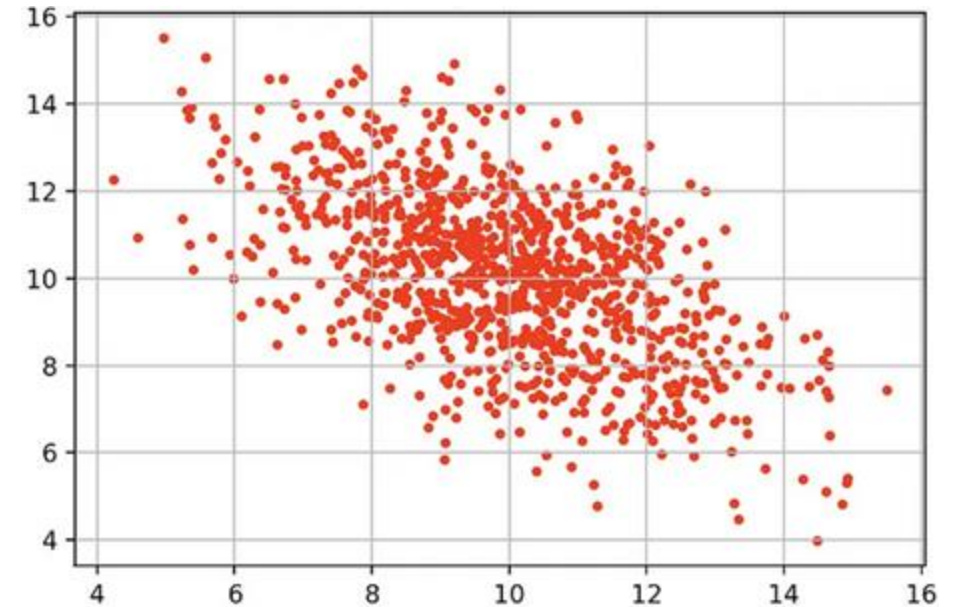
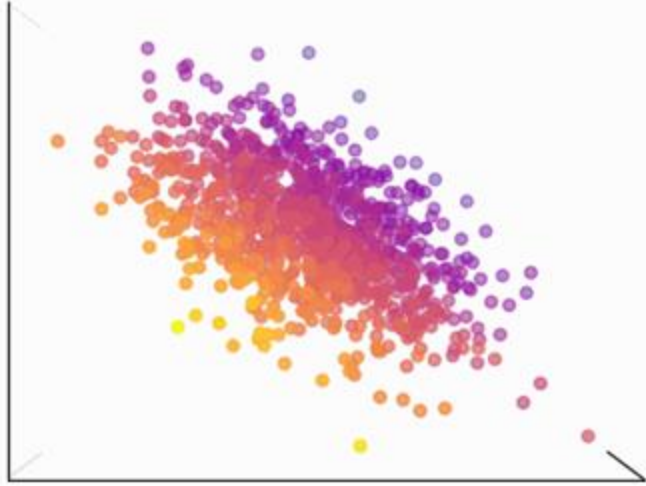
Definition

- The **curse of dimensionality** is a common problem in machine learning:
 - The performance of the model decreases as the number of features increases.
 - High dimensional data → more difficult to find a good solution.
- The main idea is to **project** first the **d-dimensional** points in a **k-dimensional** space ($k \ll d$) and then solve the problem in **low dimensions**.



Dimensionality Reduction

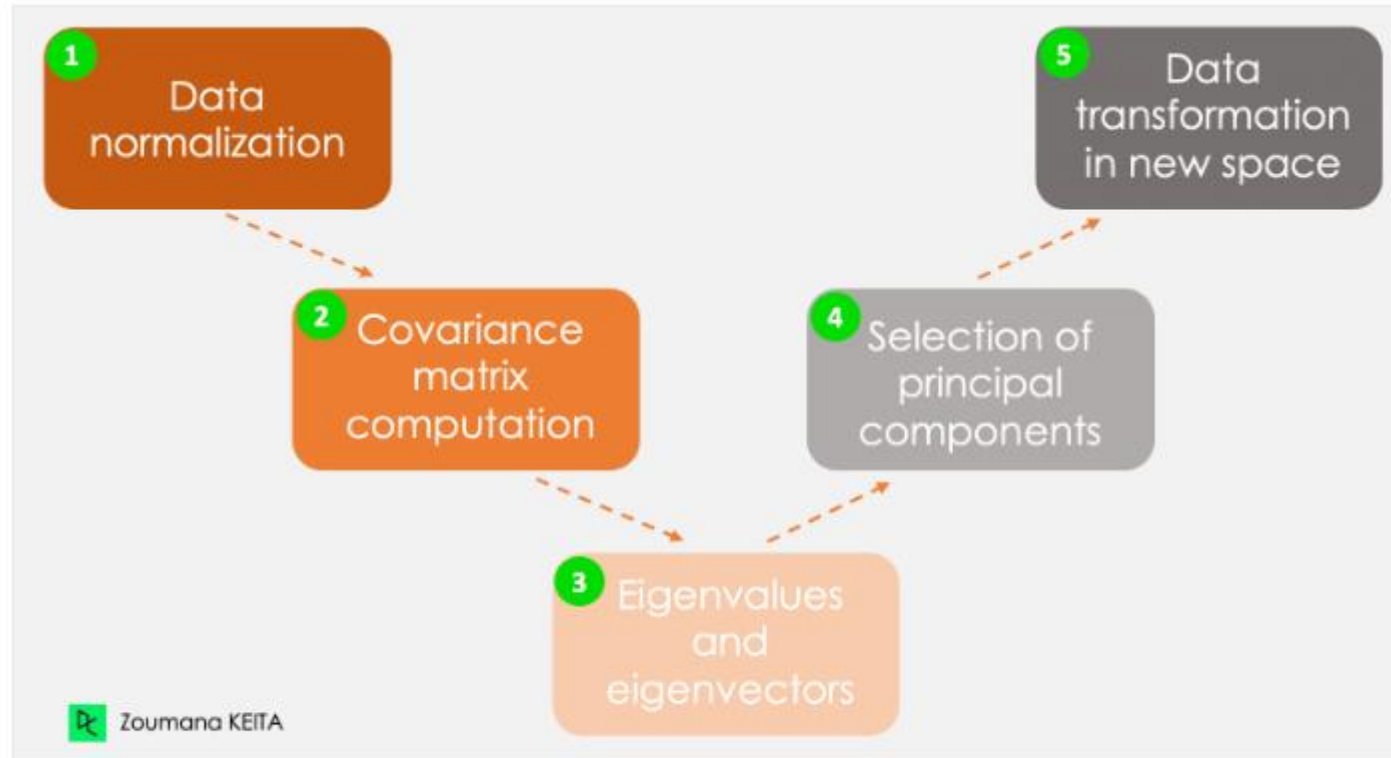
Definition



Dimensionality reduction can help to:

- Reduce the complexity of the model
- Improve the model generalization performance.

Steps of PCA



The five main steps for computing principal components

The mathematical details are covered in the
ADIF72 : Mathematics for Data Scientists module.

Dimensionality Reduction

Principal Component Analysis (PCA)

How does PCA work?

1. Standardization for each feature $X_i = \frac{x_i - \mu}{\sigma}$
2. Compute the covariance matrix to identify correlations

$$\text{Cov}(X) = \frac{1}{n-1}(X - \bar{X})(X - \bar{X})^T$$

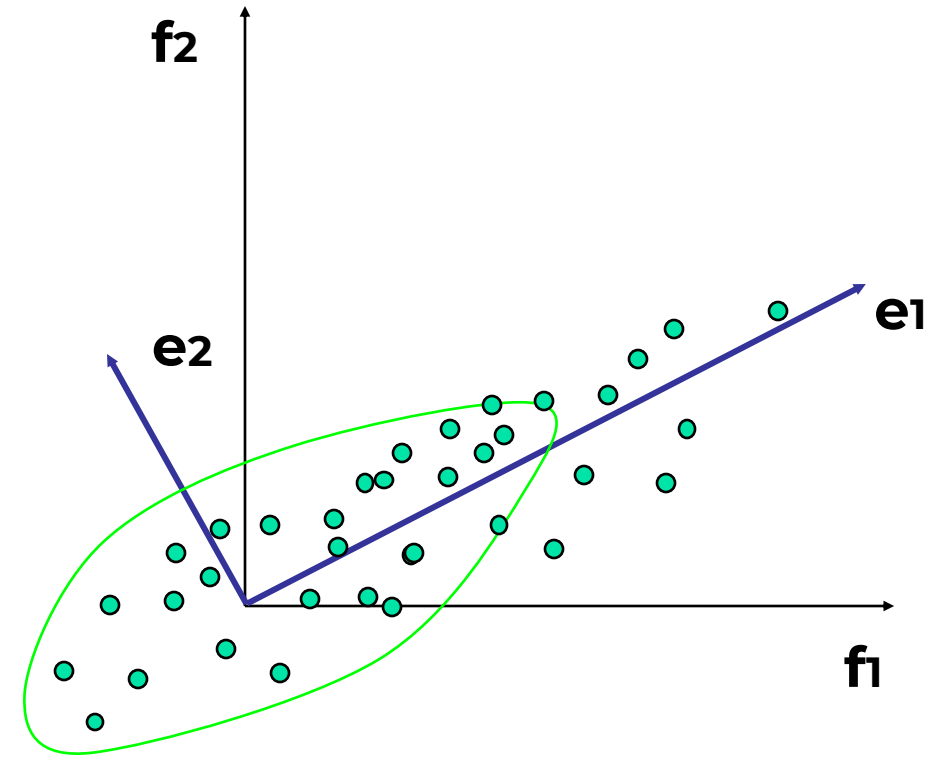
3. Compute the eigenvectors and eigenvalues
 - **Eigenvectors** represent the principal components
 - **Eigenvalues** represent the magnitude of variance.

$$\text{Cov}(X)v = \lambda v$$

$$\text{Cov}(X) = Q^T \Lambda Q$$

4. Sort the eigenvalues in descending order
 - The eigenvectors of the highest eigenvalues capture the most variance in the data
5. Choose the top k eigenvectors (principal components)
6. Transform the Data using the chosen eigenvectors

$$X_{proj} = Q^T \cdot X$$



Dimensionality Reduction

Principal Component Analysis (PCA)

Features / Samples	F1	F2	F3	F4
1	1	2	3	4
2	5	5	6	7
3	1	4	2	3
4	5	3	2	1
5	8	1	2	2

2. Compute Covariance Matrix

	F1	F2	F3	F4
F1	0.8	-0.253	0.038	-0.145
F2	-0.253	0.8	0.511	0.4945
F3	0.038	0.511	0.8	0.752
F4	-0.145	0.4945	0.752	0.8

3. Compute Eigenvalues

$$\det(\text{Cov}(X) - \lambda I) = 0$$

$$\begin{pmatrix} \lambda_1 = 2.516 \\ \lambda_2 = 1.065 \\ \lambda_3 = 0.394 \\ \lambda_4 = 0.025 \end{pmatrix}$$

1. Standardization

	F1	F2	F3	F4
mean	4	3	3	3.4
std_dev	3	1.58114	1.73205	2.30217

4. Compute Eigenvectors

$$(\text{Cov}(X) - \lambda I)v = 0$$

$$\begin{pmatrix} 0.8 - \lambda & -0.253 & 0.038 & -0.145 \\ -0.253 & 0.8 - \lambda & 0.511 & 0.4945 \\ 0.038 & 0.511 & 0.8 - \lambda & 0.752 \\ -0.145 & 0.4945 & 0.752 & 0.8 - \lambda \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \rightarrow \begin{pmatrix} v_1 & v_2 & v_3 & v_4 \\ 0.162 & -0.917 & -0.307 & 0.196 \\ -0.524 & 0.207 & -0.817 & 0.121 \\ -0.586 & -0.320 & 0.188 & -0.720 \\ -0.596 & -0.116 & 0.450 & 0.655 \end{pmatrix}$$

5. Choose the highest k eigenvalues (e.g. k=2)

Q

6. Transform Data

$$X_{proj} = Q^T \cdot X$$

Features / Samples	New F1	New F2
1	0.014	0.756
2	-2.556	-0.780
3	-0.051	1.253
4	1.014	0.0002
5	1.580	-1.230

X =

Features / Samples	F1	F2	F3	F4
1	-1	-0.63	0	0.26
2	0.33	1.265	1.732	1.564
3	-1	0.63	-0.577	-0.174
4	0.33	0	-0.577	-1.042
5	1.33	-1.265	-0.577	-0.608

Interpretation of PCA results

- **Principal components** are the new axes or directions that maximize the **variance** in the dataset.
 - PC1 typically captures the most significant underlying patterns and trends in the data.
- 1. **Explained variance**: How to visualize and interpret **eigenvalues** ? Cumulative explained variance ?
- 2. **Loading**: How to visualize and analyze the **contribution** of each feature (variable) to each PC? i.e. how the original variable does correlate with the PC (positive, negative)?
- 3. **Scores**: How to visualize and analyze the coordinates of the original data points (individuals) in the new reduced principal component space?
- 4. How to visualize and interpret both **loadings** of the variables and **scores** of the individuals using Biplots?

Tons of libraries (R and Python)



Tons of libraries for the job

Base-R (stats)

```
res.pca <- prcomp(data, scale. = TRUE)
```

```
res.pca <- princomp(data, cor = TRUE)
```

library(ade4)

```
res.pca <- dudi.pca(data, sc
```

library(ExPosition)

```
res.pca <- epPCA(data, grap
```

library(FactoMineR)

```
res.pca <- PCA(data) # Also
```

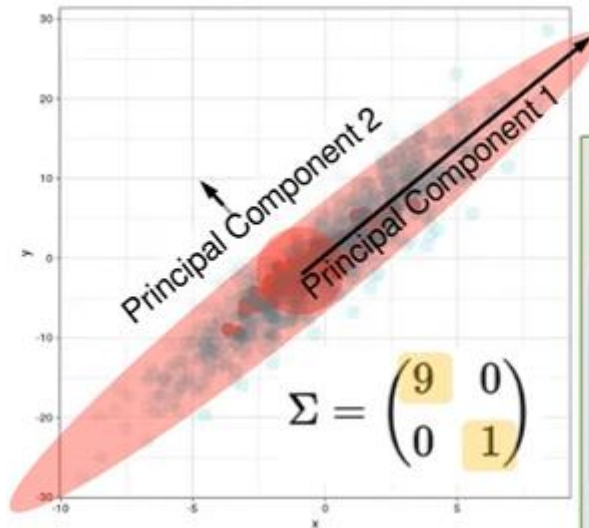
Auto-standardize data!

	V ₁	V ₂	V ₃	...	V _p	S ₁	...	S _H	C ₁	...	C _p
O ₁	Main variables and observations for PCA					Supplementary Variables (quantitative)			Supplementary Variables (qualitative)		
O ₂											
O ₃											
...											
O _p											
A ₁	Auxiliary observations										
...											
A _r											

Returns a list including:

eig	a matrix containing all the eigenvalues, the percentage of variance and the cumulative percentage of variance
var	a matrix containing all the results for the active variables (coordinates, correlation between axes, square cosine, contributions)
ind	a matrix containing all the results for the active individuals (coordinates, square cosine, contributions)
ind.sup	a list of matrices containing all the results for the supplementary individuals (coordinates, square cosine)
quanti.sup	a list of matrices containing all the results for the supplementary quantitative variables (coordinates, correlation between variables and axes)
quali.sup	a list of matrices containing all the results for the supplementary categorical variables (coordinates of each categories of each variables, v.test which is a criterion with a Normal distribution, and eta2 which is the square correlation coefficient between a qualitative variable and a dimension)

Explained variance - eigenvalues



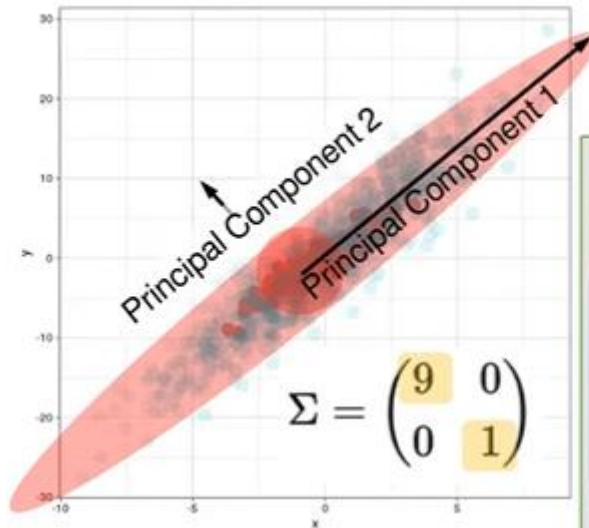
eigenvalue > 1
indicates that PCs account for more
variance than accounted
by one of the original variables
(criterion for "cutoff")

```
> res.pca <- PCA(data, scale.unit = TRUE, quali.sup = c(1,2,15), ncp = 5)
> res.pca$eig
```

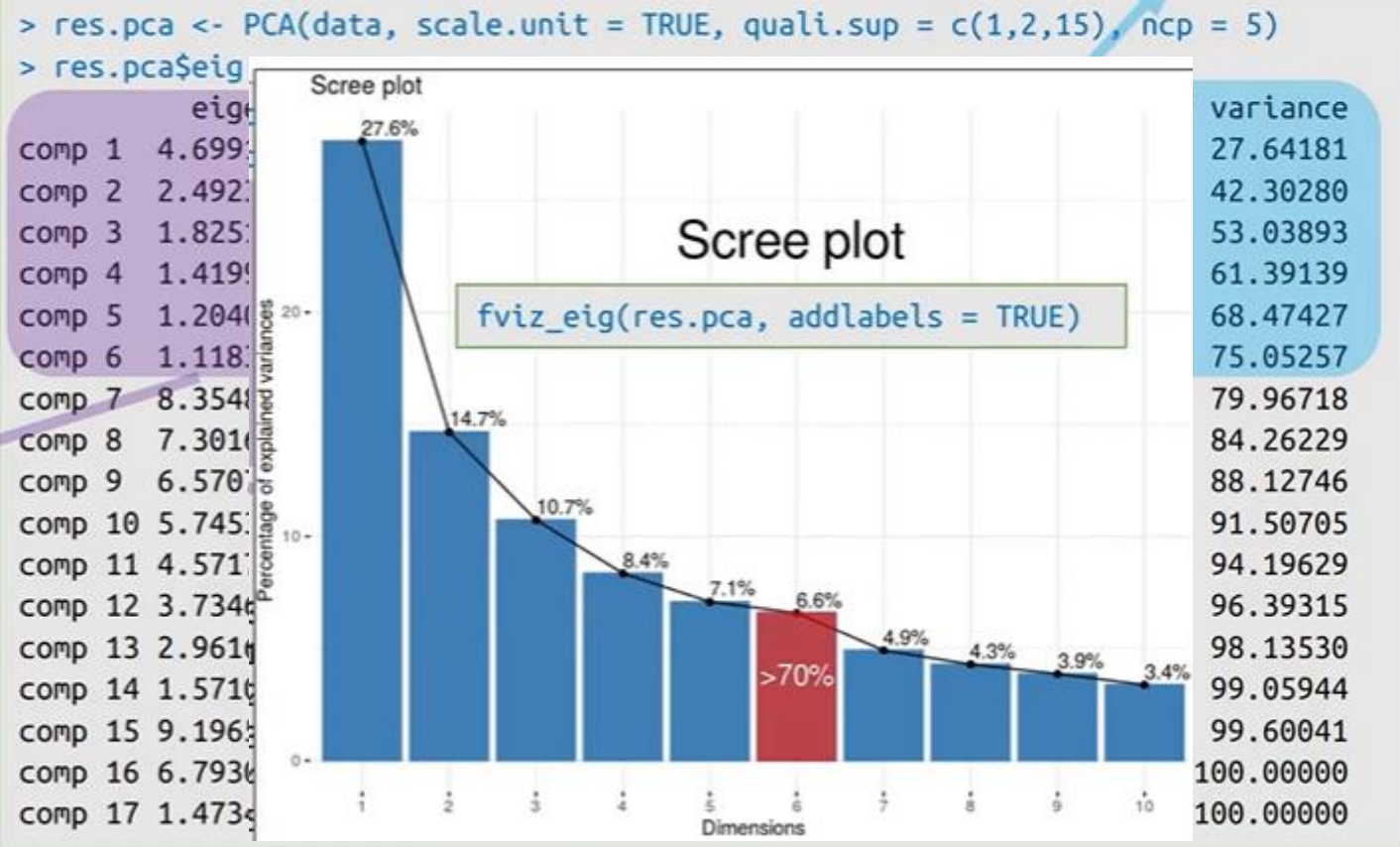
	eigenvalue	percentage of variance	cumulative percentage of variance
comp 1	4.699109e+00	2.764181e+01	27.64181
comp 2	2.492367e+00	1.466098e+01	42.30280
comp 3	1.825143e+00	1.073613e+01	53.03893
comp 4	1.419917e+00	8.352455e+00	61.39139
comp 5	1.204090e+00	7.082883e+00	68.47427
comp 6	1.118311e+00	6.578298e+00	75.05257
comp 7	8.354839e-01	4.914611e+00	79.96718
comp 8	7.301688e-01	4.295110e+00	84.26229
comp 9	6.570788e-01	3.865169e+00	88.12746
comp 10	5.745309e-01	3.379593e+00	91.50705
comp 11	4.571703e-01	2.689237e+00	94.19629
comp 12	3.734672e-01	2.196866e+00	96.39315
comp 13	2.961649e-01	1.742147e+00	98.13530
comp 14	1.571032e-01	9.241366e-01	99.05944
comp 15	9.196537e-02	5.409728e-01	99.60041
comp 16	6.793007e-02	3.995887e-01	100.00000
comp 17	1.473409e-07	8.667111e-07	100.00000

Other **criterion** is limit cumulative variance
to >70% (80%, 90%, ...)
In this example, the same "cutoff"

Explained variance - eigenvalues



eigenvalue > 1
indicates that PCs account for more variance than accounted by one of the original variables
(criterion for "cutoff")



Loadings: contribution of each var to each PC

var: variables
(not variance)

coord:
coordinates
(loadings)

```
> head(res.pca$var$coord)
```

	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
Population	0.02241625	-0.0374425	0.76590160	0.434057805	0.2096857
Area (sq. mi.)	0.03952854	0.2205003	0.63573963	0.562567298	0.2313934
Pop. Density (per sq. mi.)	0.24163728	0.1077903	-0.21420881	0.003841362	0.2706660
Coastline (coast/area ratio)	0.11934469	-0.3834396	-0.40935817	0.363530409	0.2301244
Net migration	0.15190368	0.5934147	0.12748704	-0.364475889	0.3185615
Infant mortality (per 1000 births)	-0.92159072	0.1445784	0.04632528	-0.111024580	0.1380521

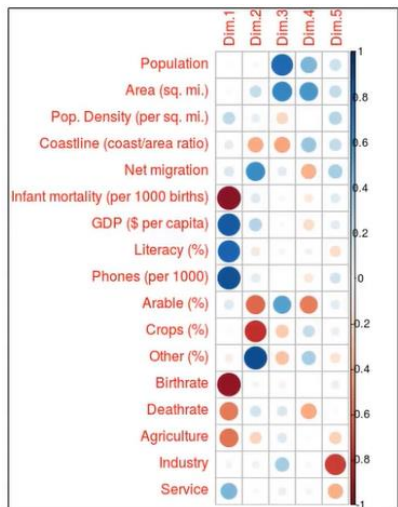
Dim._i: PC_i

Contribution of
"Population" to
PC₃ is 76.59%

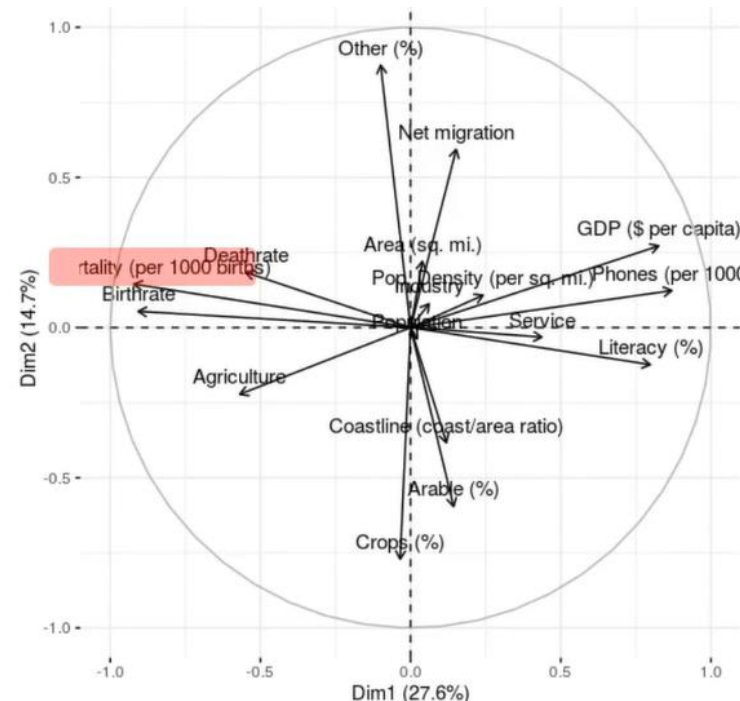
The sign is
arbitrary: doesn't
mean its bad !

Correlation circle

Correlogram



```
> fviz_pca_var(res.pca)
```



Loadings: contribution of each var to each PC (2)

var: variables
(not variance)

coord:
coordinates
(loadings)

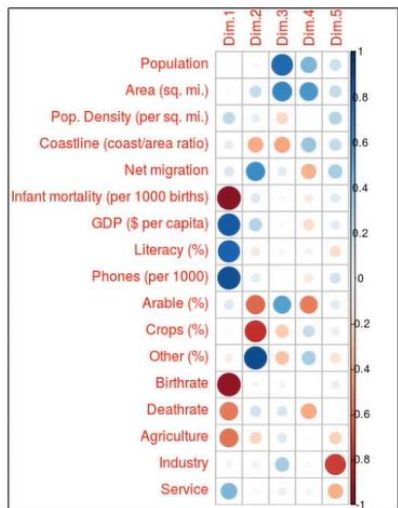
```
> head(res.pca$var$coord)
```

	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
Population	0.02241625	-0.0374425	0.76590160	0.434057805	0.2096857
Area (sq. mi.)	0.03952854	0.2205003	0.63573963	0.562567298	0.2313934
Pop. Density (per sq. mi.)	0.24163728	0.1077903	-0.21420881	0.003841362	0.2706660
Coastline (coast/area ratio)	0.11934469	-0.3834396	-0.40935817	0.363530409	0.2301244
Net migration	0.15190368	0.5934147	0.12748704	-0.364475889	0.3185615
Infant mortality (per 1000 births)	-0.92159072	0.1445784	0.04632528	-0.111024580	0.1380521

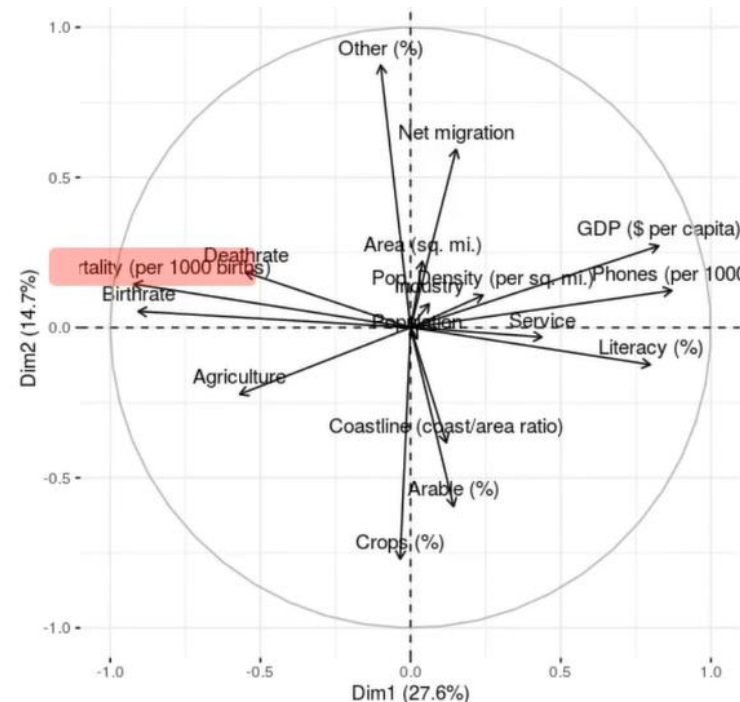
Dim._i: PC_i

Correlation circle

Correlogram



```
> fviz_pca_var(res.pca)
```



Contribution of
"Population" to
PC₃ is 76.59%

The sign is
arbitrary: doesn't
mean its bad !

Loadings: contribution of each var to each PC (3)

```
> head(res.pca$var$coord)
```

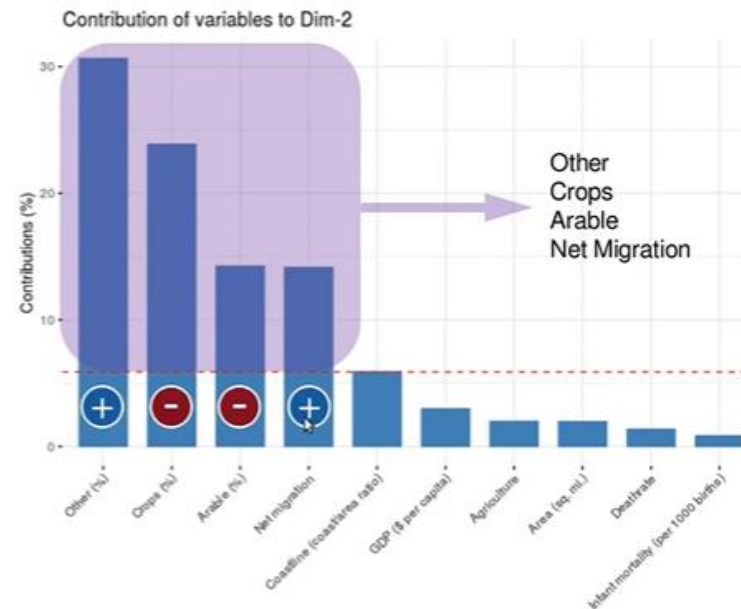
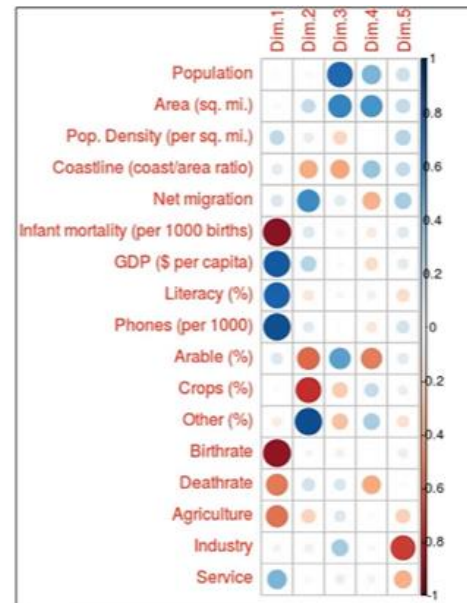
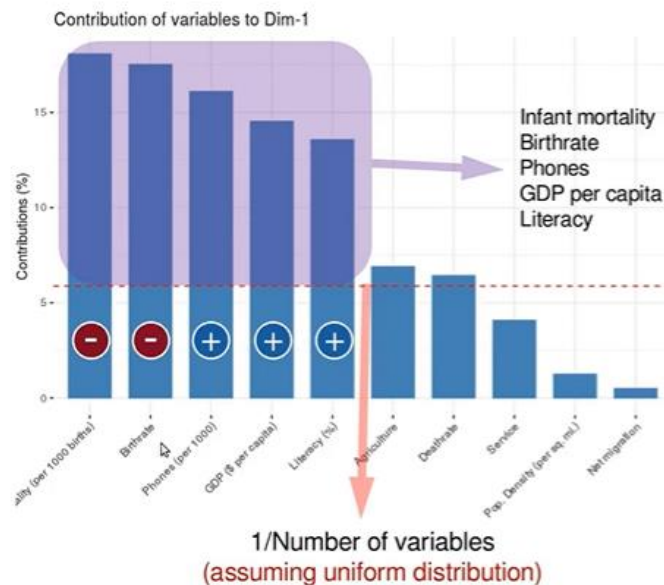
	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
Population	0.02241625	-0.0374425	0.76590160	0.434057805	0.2096857
Area (sq. mi.)	0.03952854	0.2205003	0.63573963	0.562567298	0.2313934
Pop. Density (per sq. mi.)	0.24163728	0.1077903	-0.21420881	0.003841362	0.2706660
Coastline (coast/area ratio)	0.11934469	-0.3834396	-0.40935817	0.363530409	0.2301244
Net migration	0.15190368	0.5934147	0.12748704	-0.364475889	0.3185615
Infant mortality (per 1000 births)	-0.92159072	0.1445784	0.04632528	-0.111024580	0.1380521

```
fviz_contrib(res.pca,choice = 'var',top=10)
```

```
fviz_contrib(res.pca,choice = 'var',top=10,axes = 2)
```

Top 10 var. Only PC₁

PC₁ and PC₂

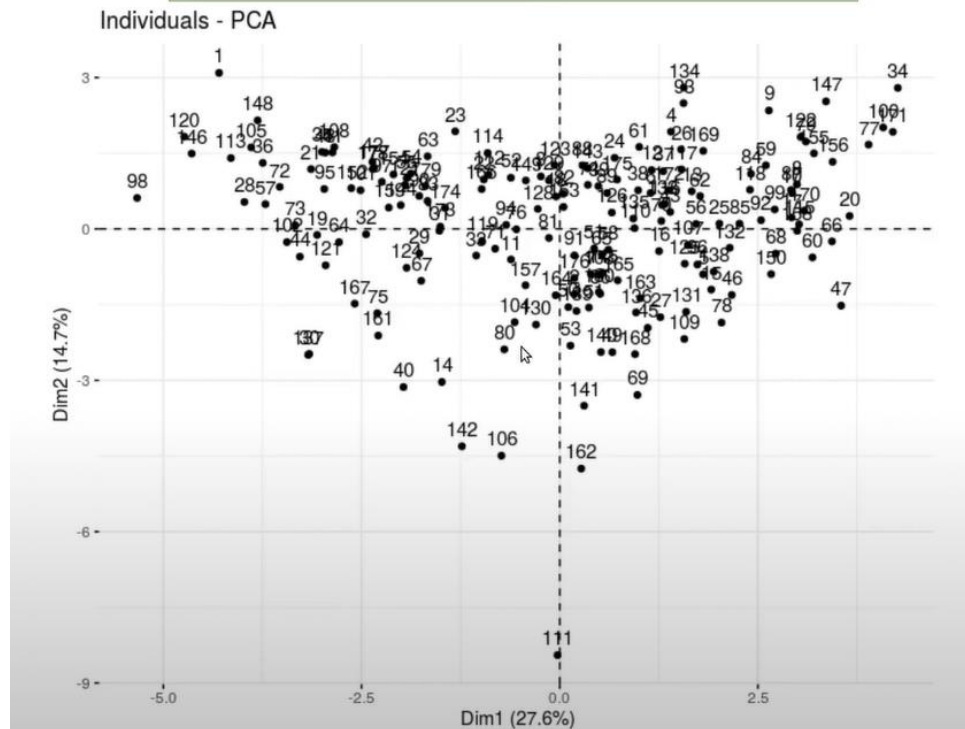


Scores: projection on the new space

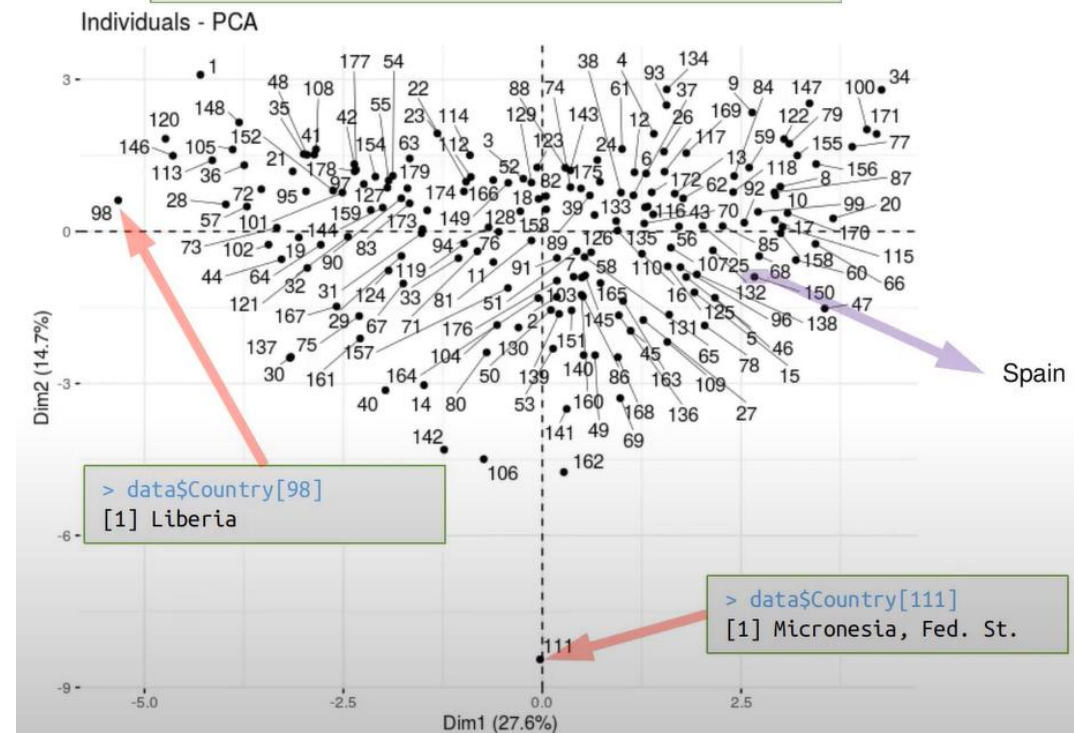
Score plots

individuals

```
fviz_pca_ind(res.pca, repel=FALSE)
```



```
fviz_pca_ind(res.pca, repel=FALSE)  
fviz_pca_ind(res.pca, repel=TRUE)
```



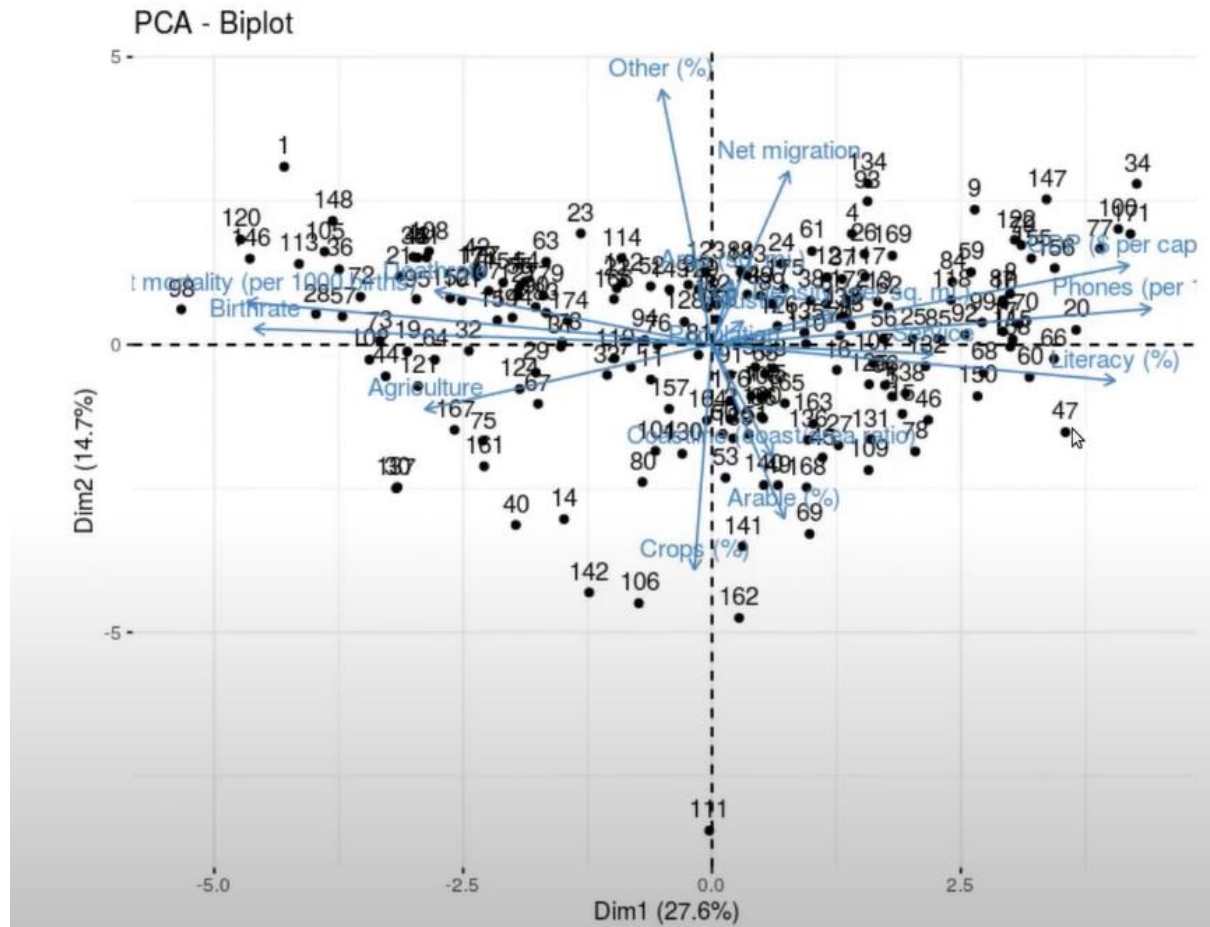
• Interpretation:

- **Data clustering:** You can use scores to visualize and analyze clusters of data points in the reduced dimension space. For example, plotting data in the PC1 vs. PC2 space can reveal groups or clusters.
- **Outliers:** Data points that lie far from others in the principal component space might be outliers.

Scores & loadings on the same graph

Biplots

```
fviz_pca_biplot(res.pca)
```



Prince



- [Prince](#) is a Python library for multivariate exploratory data analysis in Python.
- It includes a variety of methods for summarizing tabular data, including principal component analysis and correspondence analysis.
- Prince provides efficient implementations, using a [scikit-learn](#) API.
- Prince uses [Altair](#) for making charts.
- [Prince GitHub](#)
- Let's start our Lab !

Useful links

- <https://www.youtube.com/watch?v=kDbyBcDcC3I>
- Prince python library: <https://maxhalford.github.io/prince/>